

Subiect Model - Structuri de Date si TAD-uri

I. Probleme de tip grilă cu un singur răspuns corect

1. Ce structură de date este cel mai eficientă pentru implementarea unei cozi cu priorități?
A. Vector B. Stivă C. Arbore binar de căutare auto-echilibrat (ex: AVL) D. Listă simplu înlănțuită
2. Fie o listă dublu înlănțuită. Care este complexitatea timp în cel mai rău caz pentru ștergerea unui element cunoscut prin pointer?
A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$
3. Ce structură de date permite o parcurgere de tipul BFS (Breadth-First Search) într-un graf?
A. Stivă B. Coadă C. Arbore binar D. Hash table
4. Care este complexitatea medie a inserării într-un hash table cu o funcție de dispersie ideală?
A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

II. Exercițiu de programare TAD + Aplicație practică

Se dă următoarea interfață pentru un TAD `Queue<T>` (coadă), și o clasă de implementare bazată pe `List<T>` (listă):

```
template <typename T>
class Queue {
private:
    /// TODO

public:
    Queue();
    ~Queue();

    void enqueue(const T& element);
    void dequeue();
```

Subiect Model - Structuri de Date si TAD-uri

```
T& front();  
bool isEmpty() const;  
};
```

List<T> include:

- push_back, pop_front, front, empty(), etc.

Cerințe:

a) Implementați toate metodele clasei Queue folosind List<T>.

b) Simulați comportamentul unei cozi de pacienți la camera de gardă.

Fiecare pacient are:

- string nume
- int prioritate (1-10, unde 10 este urgență maximă)

Implementați:

1. structura Pacient
2. adăugarea pacienților în coadă în funcție de prioritate
3. extragerea pacienților conform priorității
4. afișarea pacientului din fața cozii (dacă există)
5. afișarea numărului de pacienți rămași